

242. Valid Anagram Easy

Acceptance: 67.6% | [LeetCode](#)

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

Example 1:

Input: `s = "anagram", t = "nagaram"`

Output: `true`

Example 2:

Input: `s = "rat", t = "car"`

Output: `false`

Constraints:

- `1 <= s.length, t.length <= 5 * 104`
- `s` and `t` consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Claude Opus Solution

Valid Anagram

1. Intuition

The **key insight** is the definition of an anagram: two strings are anagrams if they contain the **exact same characters with the exact same frequencies**, just in a different order.

This means we don't care about the *order* of characters—only the *count* of each character. So if we tally up how many times each character appears in both strings and the tallies match perfectly, they're anagrams.

Pattern: This is a classic **frequency counting** problem. Whenever you need to compare two collections regardless of order, think "count the elements."

2. Approach

1. If lengths differ → not anagrams → return False (early exit)
2. Create a frequency map for string `s` (increment counts)
3. Iterate through `t`, decrementing counts:
 - If a character isn't in the map (or its count goes negative), `t` has a character `s` doesn't have → return False
4. If all counts are zero → return True

We can fuse steps 2–4 efficiently by incrementing for `s` and decrementing for `t` in a single map.

3. Python Solution

```
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # Early exit: anagrams MUST have the same length.
        # If lengths differ, no way they're anagrams. O(1) check.
        if len(s) != len(t):
            return False

        # Counter builds a frequency dict: {char: count}
        # Two anagrams produce identical Counters.
        # Comparing Counters checks all key/value pairs match.
        return Counter(s) == Counter(t)
```

Alternative: Single fixed-size array (optimal for lowercase letters)

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # Length mismatch can never be an anagram.
        if len(s) != len(t):
            return False

        # Since input is guaranteed lowercase a-z, use a 26-slot array.
```

```
# Index 0 = 'a', index 25 = 'z'. This avoids hash overhead.
counts = [0] * 26

# Increment for chars in s, decrement for chars in t.
# ord(c) - ord('a') maps 'a'..'z' to 0..25.
for cs, ct in zip(s, t): # same length, so zip is safe
    counts[ord(cs) - ord('a')] += 1
    counts[ord(ct) - ord('a')] -= 1

# If perfectly balanced, every slot is 0 → anagram.
# Any nonzero slot means a character mismatch in frequency.
return all(c == 0 for c in counts)
```

4. Complexity Analysis

Approach	Time	Space
Counter comparison	$O(n)$	$O(1)$
Fixed array (26)	$O(n)$	$O(1)$
Sorting	$O(n \log n)$	$O(n)$

Explanation: - **Time $O(n)$:** We do a single pass over each string ($n = \text{length}$). Building/comparing counts is linear. - **Space $O(1)$:** Although a dict/array stores character counts, the alphabet is bounded (26 lowercase letters), so the space is constant regardless of input size. (If we consider a general charset of size k , it's $O(k)$.)

The **sorting approach** is slower ($O(n \log n)$) because comparing sorted strings requires sorting both first.

5. Alternative Approaches

A. Sorting

```
def isAnagram(self, s: str, t: str) -> bool:
    # If the sorted characters match, they're anagrams.
    # Simplest to write, but  $O(n \log n)$  due to sort.
    return sorted(s) == sorted(t)
```

- Pro: One line, no extra logic.
- Con: Slower; uses $O(n)$ space for sorted copies.

B. Follow-up — Unicode characters

The fixed 26-array approach **breaks** for Unicode because there are far more than 26 possible characters. The solution:

Use a hash map (dict/Counter) instead of a fixed array. A `Counter` works for *any* character set—ASCII, Unicode, emojis—because it dynamically allocates keys.

```
def isAnagram(self, s: str, t: str) -> bool:
    if len(s) != len(t):
        return False
    return Counter(s) == Counter(t) # works for any character set
```

For Unicode, be careful about **normalization** (e.g., the same visual character can have multiple code-point representations like `é` vs `e + ´`). You'd apply `unicodedata.normalize('NFC', s)` before counting to handle this robustly.

Key Takeaways

- **Anagram = same character frequencies.**
- Frequency counting (hash map / array) gives optimal $O(n)$ time.
- **Early length check** is a cheap, powerful prune.
- Choose **array** for small fixed alphabets, **hash map** for general/Unicode inputs.

Codex Solution

1. Intuition

Two strings are **anagrams** if they contain the **same characters with the same frequencies**.

For example:

```
s = "anagram"
t = "nagaram"
```

Both contain:

```
a -> 3
n -> 1
g -> 1
r -> 1
m -> 1
```

So they are anagrams.

Since the problem says both strings contain only **lowercase English letters**, we can use a fixed-size array of length 26 to count character frequencies.

Key pattern:

Count the frequency of each character in one string, subtract the frequency using the other string, then check that all counts return to zero.

2. Approach

Pseudocode

```
function isAnagram(s, t):
    if length of s != length of t:
        return false

    create count array of size 26 initialized to 0

    for each character c in s:
        increment count[c]

    for each character c in t:
        decrement count[c]

    for each value in count:
        if value is not 0:
            return false

    return true
```

Because there are only lowercase English letters, each character maps to an index:

```
'a' -> 0
'b' -> 1
'c' -> 2
...
'z' -> 25
```

We can compute that index using:

```
ord(c) - ord('a')
```

3. Python Solution

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # If lengths are different, they cannot be anagrams.
        # Anagrams must use exactly the same characters the same number of times.
```

```

if len(s) != len(t):
    return False

# Since the constraints say the strings contain only lowercase English letters,
# we only need 26 counters: one for each letter from 'a' to 'z'.
counts = [0] * 26

# Base ASCII/Unicode value for 'a'.
# This helps map characters to indices:
# 'a' -> 0, 'b' -> 1, ..., 'z' -> 25.
base = ord('a')

# Count characters in s.
for char in s:
    index = ord(char) - base
    counts[index] += 1

# Subtract character counts using t.
for char in t:
    index = ord(char) - base
    counts[index] -= 1

# If s and t are anagrams, every count should return to 0.
for count in counts:
    if count != 0:
        return False

return True

```

4. Complexity Analysis

Let n be the length of s , and since we check lengths first, t must also have length n .

Time Complexity: $O(n)$

We make three passes:

1. One pass over s
2. One pass over t
3. One pass over the fixed-size array of length 26

So the total work is:

$$O(n) + O(n) + O(26) = O(n)$$

Therefore:

$$\text{Time Complexity: } O(n)$$

Space Complexity: $O(1)$

The frequency array always has size 26, regardless of input size.

Therefore:

$$\text{Space Complexity: } O(1)$$

5. Alternative Approaches

Approach 1: Sorting

Sort both strings and compare them.

```

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return sorted(s) == sorted(t)

```

Complexity

Time Complexity: $O(n \log n)$
Space Complexity: $O(n)$

This is simple, but slower than counting because sorting costs $O(n \log n)$.

Approach 2: Hash Map / Dictionary

Use a dictionary to count characters.

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        counts = {}

        for char in s:
            counts[char] = counts.get(char, 0) + 1

        for char in t:
            if char not in counts:
                return False

            counts[char] -= 1

            if counts[char] < 0:
                return False

        return True
```

Complexity

Time Complexity: $O(n)$
Space Complexity: $O(k)$

Where k is the number of unique characters.

For lowercase English letters, $k \leq 26$, so this is effectively $O(1)$ space.

Approach 3: collections.Counter

Python provides a built-in frequency counter.

```
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return Counter(s) == Counter(t)
```

Complexity

Time Complexity: $O(n)$
Space Complexity: $O(k)$

This is very clean and Pythonic, but in interviews, implementing the counting manually is often preferred.

Follow-up: What If Inputs Contain Unicode Characters?

If the strings may contain Unicode characters, we should not use a fixed array of size 26.

Instead, use a hash map or `Counter`, because Unicode has many possible characters.

```
from collections import Counter

class Solution:
```

```
def isAnagram(self, s: str, t: str) -> bool:
    return Counter(s) == Counter(t)
```

For Unicode:

Time Complexity: $O(n)$
Space Complexity: $O(k)$

Where k is the number of distinct Unicode characters in the input.

In real-world Unicode handling, you may also need to normalize strings first. For example, some characters can be represented in multiple ways, such as "é" as a single character or as "e" plus a combining accent.

You could normalize using:

```
import unicodedata
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        s = unicodedata.normalize("NFC", s)
        t = unicodedata.normalize("NFC", t)

        return Counter(s) == Counter(t)
```